CSA0401

Operating Systems

B. Ganga Jayadev Reddy
192472248

CO2 Assessement test-1

30/7/26

1. CPU scheduling - waiting time - turn around time analysis
a set of process arrives at time '0' with burst times:

(21)

Process	burst time
P_1	10
P_2	4
P_3	6
P_4	2

a) Calculate waiting time and turnaround time for each Process using SJF.
b) compute the average waiting time and average turn around time
c) Interpret whether SJF improves cpu utilization compared to FCFS for this workload.

2. Round Robin scheduling - Time quantum impact
consider the following processes

Process	burst time
P_1	20
P_2	5
P_3	13
P_4	8

quantum = 4MS

a) Construct the gantt chart for round robin
b) Calaulate waiting time and turnaround time for each process.
c) Analyze how changing the quantum to 10ms would affect system performance

3. Dead lock safety check using bankers algorithm.

A system has 3 resource type (A, B, C) total resources

$A = 10, B = 5, C = 7$

the allocation and max need matrices are

allocation matrix

Process	A	B	C
P_1	0	1	0
P_2	2	0	0
P_3	3	0	2
P_4	2	1	1
P_5	0	0	2

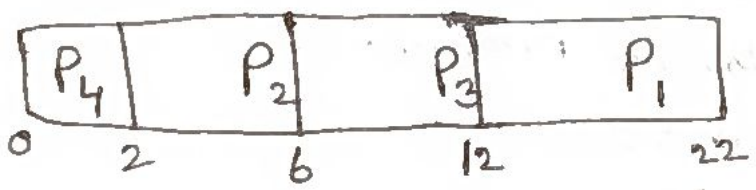
a) Calculate the need matrix

b) Determine if the system is in a safe state using Bankers algorithm

c) Provide a valid safe sequence if it exists.

1. a. SJF :-

Gantt chart



Process	Burst time	turn around time	waiting time
P_4	2	2	0
P_2	4	6	2
P_3	6	12	6
P_1	10	22	12

b. avg. turn around time $= \frac{2+6+12+22}{4}$

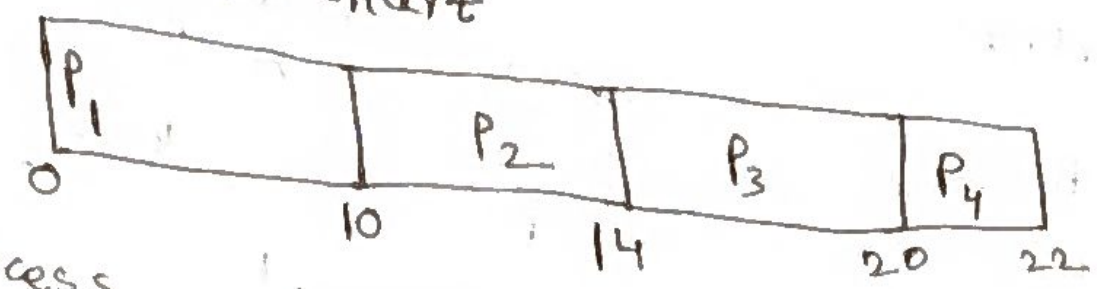
$= 10.5$

avg. waiting time $= \frac{0+2+6+12}{4}$

$= 5$

c. FCFS :-

Gantt chart



Process	burst time	turn around time	waiting time
P_1	10	10	0
P_2	4	14	10
P_3	6	20	14
P_4	2	22	20

avg tAt = $\frac{10+14+20+22}{4}$ = 16.5

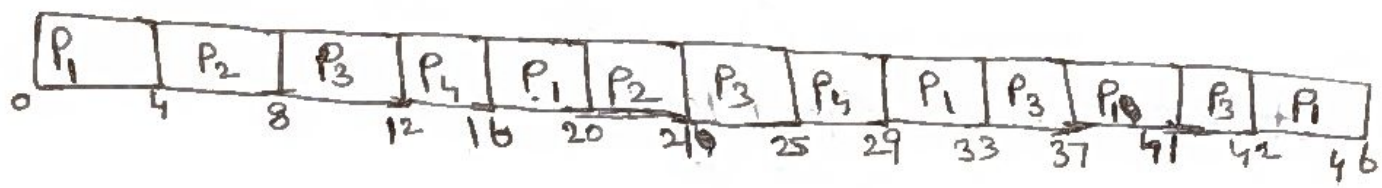
avg. wt = $\frac{0+10+14+20}{4}$ = 11

∴ SJF improves cpu utilization compared to FCFS

2. a. Round robin

quantum = 4

Gantt chart



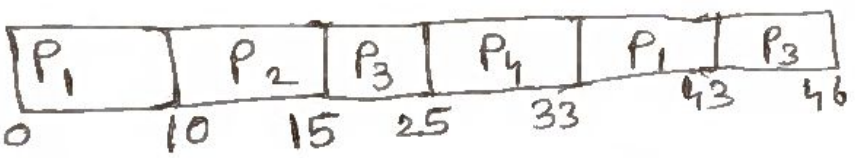
b.

Process	burst time	TAT	WT
P_1	20	46	26
P_2	5	21	16
P_3	13	42	29
P_4	8	29	21

avg TAT = $\frac{46+21+42+29}{4}$ = 39.5

avg WT = $\frac{26+16+29+21}{4}$ = 23

c. quantum change = 10



∴ changing quantum to 10ms
∴ may improves cpu utilization

Process	burst time	TAT	WT
P_1	20	43	23
P_2	5	15	10
P_3	13	46	33
P_4	8	33	25

avg TAT = $\frac{43+15+46+33}{4}$ = 34.25

avg WT = $\frac{23+10+33+25}{4}$ = 22.75

3. a. Available matrix

Process	A	B	C
P_1	10	4	7
P_2	8	5	7
P_3	7	5	5
P_4	8	4	6
P_5	10	5	5

Need matrix

Process	A	B	C
P_1	7	4	3
P_2	1	2	2
P_3	6	0	0
P_4	2	1	1
P_5	5	3	1

b. at P_1 $(7,4,3) \leq (10,4,7)$

new arrival $= (10,5,7)$

at P_2 $(1,2,2) \leq (10,5,7)$

new arr. $= (12,5,7)$

at P_3 $(6,0,0) \leq (12,5,7)$

new arr. $= (15,5,9)$

at P_4 $(2,1,1) \leq (15,5,9)$

new arr $= (17,6,10)$

at P_5 $(5,3,1) \leq (17,6,10)$

new arr $= (17,6,12)$

safest route is

$P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4 \rightarrow 5$

c) Both Given and safest is same

~~P_1, P_2, P_3, P_4, P_5~~

4. Given

- P_1 is holding R_1 & waiting for R_2
- P_2 is holding R_2 & waiting for R_3
- P_3 is holding & waiting for R_1

(a) In RAG

Nodes :

Process = P_1 P_2 P_3

Resources = R_1 R_2 R_3

Edges

$R_1 \rightarrow P_1$ (P_1 is holding R_1)

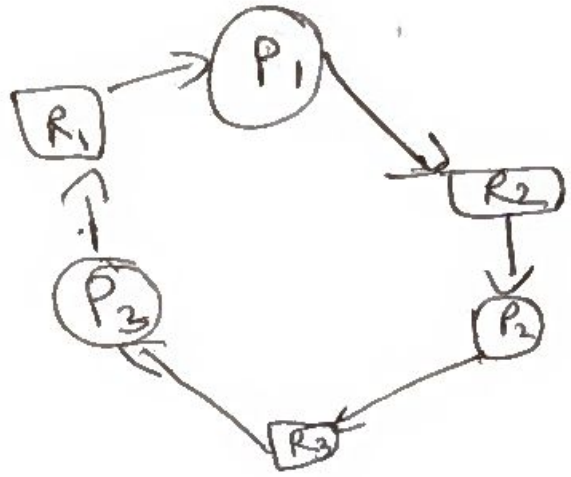
$P_1 \rightarrow R_2$ (P_1 waiting for R_2)

$R_2 \rightarrow P_2$ (P_2 holding R_2)

$P_2 \rightarrow R_3$ (P_2 waiting for R_3)

$R_3 \rightarrow P_3$ (P_3 holding R_3)

$P_3 \Rightarrow R_1$ (P_3 waiting for R_1)



(b) System is deadlock :-

cycle $P_1 \rightarrow R_2 \rightarrow P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_1 \rightarrow P_1$

(c) Strategy :-

Deadlock detection & recovery

Justification :-

- system can allow resource allocation freely & detect deadlock
- After detection, recover by terminating one process, then reclaim its resources & proceed.

5. CPU formula =

$$U = 1 - (p^n)$$

Here

$p = 80\% = 0.8$

(a) CPU utilization for $n = 4$

$$U = 1 - (0.8)^4$$
$$= 1 - 0.4096$$
$$= 0.5904$$

CPU utilization = 59.04%.

(b) $n = 6$

$$U = 1 - (0.8)^6$$
$$= 1 - 0.262144$$
$$= 0.737856$$

CPU utilization = 73.79%

(c)

* when degree of multiprogramming increases from 4 to 6, CPU utilization ~~from~~ increases from 59.04% to 73.79%
* This means CPU stays busy more often and spends less time idle.
* So higher degree of multiprogramming ($n = 6$) improves system performance.